

**LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
TUGAS 2**



Tugas 2

Oleh:

Aufa Thariq Akram

2510817310014

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
MEI 2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
TUGAS 2

Laporan Praktikum Algoritma & Struktur Data Tugas 2 ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Aufa Thariq Akram
NIM : 2510817310014

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Fadhil Syahdama Mahatma Putra
NIM. 2410817210026

Andreyan Rizky Baskara, S.Kom., M.Kom
NIP. 199307032019031011

DAFTAR ISI

LEMBAR PENGESAHAN	i
DAFTAR ISI.....	ii
DAFTAR GAMBAR	iii
DAFTAR TABEL.....	iv
TUGAS 2	1
A. Source Code	2
B. Output Program	13
LINK GITHUB	16

DAFTAR GAMBAR

Gambar 1 Screenshot Output Program Tugas 2 Modifikasi 1	13
Gambar 2 Screenshot Output Program Tugas 2 Modifikasi 2	14
Gambar 3 Screenshot Output Program Tugas 2 Modifikasi 3	14
Gambar 4 Screenshot Output Program Tugas 2 Modifikasi 4	15

DAFTAR TABEL

Tabel 1. Source Code Tugas 2	2
------------------------------------	---

TUGAS 2

Berdasarkan kode yang sudah dibuat pada modul 4 lakukan modifikasi pada kode berikut :

1. Modifikasi semua fungsi penambahan agar :

- Dapat menerima beberapa data dalam satu baris
- Data dipisahkan menggunakan spasi
- Urutan data dalam linked list harus sama dengan urutan input
- Jika menggunakan tambah depan maka data harus tetap tersusun sesuai urutan input
- (tidak terbalik)
- Jika linked list sudah memiliki data, maka :
 - Tambah depan menambahkan data sebelum data lama
 - Tambah belakang menambahkan data setelah data lama

2. Buat fungsi hapus berdasarkan target dengan ketentuan :

- Menghapus semua node yang memiliki data yang sama dengan target
- Jika data ditemukan lebih dari satu, semua harus dihapus
- Jika tidak ditemukan tampilkan pesan

3. Buat fungsi clear dengan ketentuan :

- Menghapus node secara bergantian dari depan dan belakang
- Urutan penghapusan dimulai dari depan
- Nomor urutan penghapusan dimulai dari 1
- Setiap penghapusan harus menampilkan isi node yang dihapus dan urutan penghapusan

4. modifikasi fungsi hapusBelakangH() dan hapusBelakangHT() agar :

- Dapat menghapus node ke-N dari belakang
- Jika N terlalu besar dari jumlah node maka gunakan sistem mundur

A. Source Code

Tabel 1. Source Code Tugas 2

1	#include <iostream>
2	#include <stdlib.h>
3	#include <termios.h>
4	#include <unistd.h>
5	#include <sstream>
6	#include <string>
7	
8	using namespace std;
9	
10	int getch() {
11	struct termios oldt, newt;
12	int ch;
13	tcgetattr(STDIN_FILENO, &oldt);
14	newt = oldt;
15	newt.c_lflag &= ~(ICANON ECHO);
16	tcsetattr(STDIN_FILENO, TCSANOW, &newt);
17	ch = getchar();
18	tcsetattr(STDIN_FILENO, TCSANOW, &oldt);
19	return ch;
20	}
21	
22	typedef struct TNode {
23	string data;
24	TNode *next;
25	TNode *prev;
26	};
27	
28	TNode *head, *tail;
29	
30	int pil, menu;
31	string dataBaru;
32	
33	void initH();
34	void initHT();
35	int isEmptyH();
36	int isEmptyHT();
37	
38	void tambahDepanH();
39	void tambahDepanHT();
40	void tambahBelakangH();
41	void tambahBelakangHT();
42	void hapusDepanH();

```

43 void hapusDepanHT();
44 void hapusBelakangH();
45 void hapusBelakangHT();
46 void hapusTargetH();
47 void hapusTargetHT();
48 void tampilkanH();
49 void tampilkanHT();
50 void clearH();
51 void clearHT();
52
53 int main()
54 {
55     menu:
56     cout<<"Double Linked List Non Circular
(DLLNC) "<<endl;
57
58     cout<<"===== "<<en
dl;
59     cout<<"Silahkan pilih program DLLNC yang ingin
dijalankan!"<<endl;
60     cout<<"1. DLLNC dengan Head"<<endl;
61     cout<<"2. DLLNC dengan Head dan Tail"<<endl;
62     cout<<"3. Quit"<<endl;
63     cout<<"Pilihan : ";
64     cin>>menu;
65     system("clear");
66     if(menu==1){
67         do {
68             cout<<"Double Linked List Non Circular
(DLLNC) (Head) "<<endl;
69
70             cout<<"=====
==== "<<endl;
71             cout<<"1. Tambah Depan"<<endl;
72             cout<<"2. Tambah Belakang"<<endl;
73             cout<<"3. Tampilkan Data"<<endl;
74             cout<<"4. Hapus Depan"<<endl;
75             cout<<"5. Hapus Belakang (Hapus ke-
N) "<<endl;
76             cout<<"6. Hapus Target"<<endl;
77             cout<<"7. Reset (Clear
Bergantian) "<<endl;
78             cout<<"8. Kembali ke Menu"<<endl;
79             cout<<"Pilihan : ";
80             cin>>pil;

```


79	
80	switch(pil) {
81	case 1: tambahDepanH(); break;
82	case 2: tambahBelakangH(); break;
83	case 3: tampilkanH(); break;
84	case 4: hapusDepanH(); break;
85	case 5: hapusBelakangH(); break;
86	case 6: hapusTargetH(); break;
87	case 7: clearH(); break;
88	case 8: system("clear"); goto menu;
89	default: break;
90	}
91	
92	cout<<"\n<press any key to
	continue>"<<endl;
93	getch();
94	system("clear");
95	
96	} while (pil!=8);
97	} else if(menu==2){
98	do {
99	cout<<"Double Linked List Non Circular
	(DLLNC) (Head dan Tail)"<<endl;
100	cout<<"=====
	===="<<endl;
101	cout<<"1. Tambah Depan"<<endl;
102	cout<<"2. Tambah Belakang"<<endl;
103	cout<<"3. Tampilkan Data"<<endl;
104	cout<<"4. Hapus Depan"<<endl;
105	cout<<"5. Hapus Belakang (Hapus ke-
	N)"<<endl;
106	cout<<"6. Hapus Target"<<endl;
107	cout<<"7. Reset (Clear
	Bergantian)"<<endl;
108	cout<<"8. Kembali ke Menu"<<endl;
109	cout<<"Pilihan : ";
110	cin>>pil;
111	
112	switch(pil) {
113	case 1: tambahDepanHT(); break;
114	case 2: tambahBelakangHT(); break;
115	case 3: tampilkanHT(); break;
116	case 4: hapusDepanHT(); break;
117	case 5: hapusBelakangHT(); break;

```

118         case 6: hapusTargetHT(); break;
119         case 7: clearHT(); break;
120         case 8: system("clear"); goto menu;
121         default: break;
122     }
123
124     cout<<"\npress any key to
continue"<<endl;
125     getch();
126     system("clear");
127
128     } while (pil!=8);
129     } else {
130         cout<<"\nTERIMA KASIH"<<endl;
131         cout<<"Program was made by Aufa Thariq
Akram (2510817310014)."<<endl;
132     }
133 }
134
135 void initH() { head = NULL; }
136 void initHT() { head = NULL; tail = NULL; }
137 int isEmptyH() { if(head == NULL) return 1; else
return 0; }
138 int isEmptyHT() { if(tail == NULL) return 1; else
return 0; }
139
140 void tambahDepanH() {
141     cout<<"Masukkan data (pisahkan spasi untuk
multi-data) : ";
142     cin >> ws; getline(cin, dataBaru);
143     stringstream ss(dataBaru); string token;
144     TNode *newHead = NULL, *newTail = NULL;
145
146     while(ss >> token) {
147         TNode *baru = new TNode;
148         baru->data = token; baru->next = NULL;
baru->prev = NULL;
149         if(!newHead) { newHead = newTail = baru; }
150         else { newTail->next = baru; baru->prev =
newTail; newTail = baru; }
151     }
152
153     if(newHead != NULL) {
154         if (isEmptyH() == 1) { head = newHead; }

```

```

155         else { newTail->next = head; head->prev =
newTail; head = newHead; }
156         cout << "Data berhasil dimasukkan di
bagian depan sesuai urutan input.\n";
157     }
158 }
159
160 void tambahDepanHT() {
161     cout<<"Masukkan data (pisahkan spasi untuk
multi-data) : ";
162     cin >> ws; getline(cin, dataBaru);
163     stringstream ss(dataBaru); string token;
164     TNode *newHead = NULL, *newTail = NULL;
165
166     while(ss >> token) {
167         TNode *baru = new TNode;
168         baru->data = token; baru->next = NULL;
baru->prev = NULL;
169         if(!newHead) { newHead = newTail = baru; }
170         else { newTail->next = baru; baru->prev =
newTail; newTail = baru; }
171     }
172
173     if(newHead != NULL) {
174         if(isEmptyHT() == 1) { head = newHead;
tail = newTail; }
175         else { newTail->next = head; head->prev =
newTail; head = newHead; }
176         cout << "Data berhasil dimasukkan di
bagian depan sesuai urutan input.\n";
177     }
178 }
179
180 void tambahBelakangH() {
181     cout<<"Masukkan data (pisahkan spasi untuk
multi-data) : ";
182     cin >> ws; getline(cin, dataBaru);
183     stringstream ss(dataBaru); string token;
184     TNode *newHead = NULL, *newTail = NULL;
185
186     while(ss >> token) {
187         TNode *baru = new TNode;
188         baru->data = token; baru->next = NULL;
baru->prev = NULL;
189         if(!newHead) { newHead = newTail = baru; }

```

```

190         else { newTail->next = baru; baru->prev =
newTail; newTail = baru; }
191     }
192
193     if(newHead != NULL) {
194         if(isEmptyH() == 1) { head = newHead; }
195         else {
196             TNode *bantu = head;
197             while(bantu->next != NULL) bantu =
bantu->next;
198             bantu->next = newHead;
199             newHead->prev = bantu;
200         }
201         cout << "Data berhasil dimasukkan di
bagian belakang sesuai urutan input.\n";
202     }
203 }
204
205 void tambahBelakangHT() {
206     cout<<"Masukkan data (pisahkan spasi untuk
multi-data) : ";
207     cin >> ws; getline(cin, dataBaru);
208     stringstream ss(dataBaru); string token;
209     TNode *newHead = NULL, *newTail = NULL;
210
211     while(ss >> token) {
212         TNode *baru = new TNode;
213         baru->data = token; baru->next = NULL;
baru->prev = NULL;
214         if(!newHead) { newHead = newTail = baru; }
215         else { newTail->next = baru; baru->prev =
newTail; newTail = baru; }
216     }
217
218     if(newHead != NULL) {
219         if(isEmptyHT() == 1) { head = newHead;
tail = newTail; }
220         else { tail->next = newHead; newHead->prev
= tail; tail = newTail; }
221         cout << "Data berhasil dimasukkan di
bagian belakang sesuai urutan input.\n";
222     }
223 }
224
225 void tampilkanH() {

```

```

226     TNode *bantu = head;
227     if(isEmptyH() == 0) {
228         while(bantu != NULL) {
229             cout<<bantu->data<<' ';
230             bantu = bantu->next;
231         }
232         cout<<endl;
233     } else cout<<"Tidak terdapat data pada Linked
List.\n";
234 }
235
236 void tampilkanHT() {
237     TNode *bantu = head;
238     if(isEmptyHT() == 0) {
239         while(bantu != NULL) {
240             cout<<bantu->data<<' ';
241             bantu = bantu->next;
242         }
243         cout<<endl;
244     } else cout<<"Tidak terdapat data pada Linked
List.\n";
245 }
246
247 void hapusDepanH() {
248     if(isEmptyH() == 0) {
249         TNode *hapus = head; string data = hapus-
>data;
250         if(head->next != NULL) { head = head-
>next; head->prev = NULL; }
251         else { initH(); }
252         delete hapus;
253         cout<<"Data \""<<data<<"\" yang berada di
depan telah berhasil dihapus.\n";
254     } else cout<<"Tidak terdapat data pada Linked
List.\n";
255 }
256
257 void hapusDepanHT() {
258     if(isEmptyHT() == 0) {
259         TNode *hapus = head; string data = hapus-
>data;
260         if(head->next != NULL) { head = head-
>next; head->prev = NULL; }
261         else { initHT(); }
262         delete hapus;

```

```

263         cout<<"Data \""<<data<<"\" yang berada di
depan telah berhasil dihapus.\n";
264     } else cout<<"Tidak terdapat data pada Linked
List.\n";
265 }
266
267 void hapusBelakangH() {
268     if(isEmptyH() == 1) { cout<<"Tidak terdapat
data pada Linked List.\n"; return; }
269     int n; cout<<"Masukkan N (hapus node ke-N dari
belakang) : "; cin>>n;
270
271     int count = 0; TNode* bantu = head; TNode*
tempTail = NULL;
272     while(bantu != NULL) { count++; if(bantu->next
== NULL) tempTail = bantu; bantu = bantu->next; }
273
274     int pos = n % count;
275     if(pos == 0) pos = count;
276
277     bantu = tempTail;
278     for(int i = 1; i < pos; i++) bantu = bantu-
>prev;
279
280     string dataHapus = bantu->data;
281     if(bantu == head && bantu->next == NULL) {
head = NULL; }
282     else if(bantu == head) { head = head->next;
head->prev = NULL; }
283     else if(bantu->next == NULL) { bantu->prev-
>next = NULL; }
284     else { bantu->prev->next = bantu->next; bantu-
>next->prev = bantu->prev; }
285
286     delete bantu;
287     cout<<"Node ke-"<<n<<" dari belakang
(\"\"<<dataHapus<<"\") berhasil dihapus.\n";
288 }
289
290 void hapusBelakangHT() {
291     if(isEmptyHT() == 1) { cout<<"Tidak terdapat
data pada Linked List.\n"; return; }
292     int n; cout<<"Masukkan N (hapus node ke-N dari
belakang) : "; cin>>n;
293

```

```

294     int count = 0; TNode *bantu = head;
295     while(bantu != NULL) { count++; bantu = bantu-
>next; }
296
297     int pos = n % count;
298     if(pos == 0) pos = count;
299
300     bantu = tail;
301     for(int i = 1; i < pos; i++) bantu = bantu-
>prev;
302
303     string dataHapus = bantu->data;
304     if(bantu == head && bantu == tail) { head =
tail = NULL; }
305     else if(bantu == head) { head = head->next;
head->prev = NULL; }
306     else if(bantu == tail) { tail = tail->prev;
tail->next = NULL; }
307     else { bantu->prev->next = bantu->next; bantu-
>next->prev = bantu->prev; }
308
309     delete bantu;
310     cout<<"Node ke-"<<n<<" dari belakang
(\"\"<<dataHapus<<"\") berhasil dihapus.\n";
311 }
312
313 void hapusTargetH() {
314     if(isEmptyH() == 1) { cout << "Linked list
kosong.\n"; return; }
315     cout << "Masukkan data target yang ingin
dihapus: ";
316     string target; cin >> ws; getline(cin,
target);
317
318     TNode *bantu = head; int countDeleted = 0;
319     while(bantu != NULL) {
320         if(bantu->data == target) {
321             TNode *hapus = bantu;
322             if(bantu == head) {
323                 head = head->next;
324                 if(head != NULL) head->prev =
NULL;
325                 bantu = head;
326             } else {
327                 bantu->prev->next = bantu->next;

```

```

328         if(bantu->next != NULL) bantu-
>next->prev = bantu->prev;
329         bantu = bantu->next;
330     }
331     delete hapus; countDeleted++;
332     } else { bantu = bantu->next; }
333 }
334
335     if(countDeleted > 0) cout << "Berhasil
menghapus " << countDeleted << " node dengan data
\"\" << target << "\".\n";
336     else cout << "Pesan: Data \"\" << target << "\"
tidak ditemukan.\n";
337 }
338
339 void hapusTargetHT() {
340     if(isEmptyHT() == 1) { cout << "Linked list
kosong.\n"; return; }
341     cout << "Masukkan data target yang ingin
dihapus: ";
342     string target; cin >> ws; getline(cin,
target);
343
344     TNode *bantu = head; int countDeleted = 0;
345     while(bantu != NULL) {
346         if(bantu->data == target) {
347             TNode *hapus = bantu;
348             if(bantu == head && bantu == tail) {
head = tail = NULL; bantu = NULL; }
349             else if(bantu == head) { head = head-
>next; head->prev = NULL; bantu = head; }
350             else if(bantu == tail) { tail = tail-
>prev; tail->next = NULL; bantu = NULL; }
351             else { bantu->prev->next = bantu-
>next; bantu->next->prev = bantu->prev; bantu =
bantu->next; }
352             delete hapus; countDeleted++;
353             } else { bantu = bantu->next; }
354     }
355
356     if(countDeleted > 0) cout << "Berhasil
menghapus " << countDeleted << " node dengan data
\"\" << target << "\".\n";
357     else cout << "Pesan: Data \"\" << target << "\"
tidak ditemukan.\n";

```



```

358 }
359
360 void clearH() {
361     int urutan = 1;
362     while(head != NULL) {
363         TNode *hapusDepan = head;
364         cout << "Penghapusan urutan " << urutan++
<< " : " << hapusDepan->data << "\n";
365         if(head->next == NULL) { head = NULL;
delete hapusDepan; break; }
366         head = head->next; head->prev = NULL;
delete hapusDepan;
367
368         TNode *hapusBelakang = head;
369         while(hapusBelakang->next != NULL)
hapusBelakang = hapusBelakang->next;
370         cout << "Penghapusan urutan " << urutan++
<< " : " << hapusBelakang->data << "\n";
371         if(hapusBelakang->prev == NULL) { head =
NULL; }
372         else { hapusBelakang->prev->next = NULL; }
373         delete hapusBelakang;
374     }
375     cout << "\nSeluruh data telah dibersihkan
secara bergantian.\n";
376 }
377
378 void clearHT() {
379     int urutan = 1;
380     while(head != NULL) {
381         TNode *hapusDepan = head;
382         cout << "Penghapusan urutan " << urutan++
<< " : " << hapusDepan->data << "\n";
383         if(head == tail) { head = tail = NULL;
delete hapusDepan; break; }
384         head = head->next; head->prev = NULL;
delete hapusDepan;
385
386         TNode *hapusBelakang = tail;
387         cout << "Penghapusan urutan " << urutan++
<< " : " << hapusBelakang->data << "\n";
388         if(head == tail) { head = tail = NULL;
delete hapusBelakang; break; }
389         tail = tail->prev; tail->next = NULL;
delete hapusBelakang;

```

390	}
391	cout << "\nSeluruh data telah dibersihkan secara bergantian.\n";
392	}

B. Output Program

```

Double Linked List Non Circular (DLLNC) (Head)
=====
1. Tambah Depan
2. Tambah Belakang
3. Tampilkan Data
4. Hapus Depan
5. Hapus Belakang (Hapus ke-N)
6. Hapus Target
7. Reset (Clear Bergantian)
8. Kembali ke Menu
Pilihan : 3
4 5 6 1 2 3
<press any key to continue>
Double Linked List Non Circular (DLLNC) (Head)
=====
1. Tambah Depan
2. Tambah Belakang
3. Tampilkan Data
4. Hapus Depan
5. Hapus Belakang (Hapus ke-N)
6. Hapus Target
7. Reset (Clear Bergantian)
8. Kembali ke Menu
Pilihan :

```

Gambar 1 Screenshot Output Program Tugas 2 Modifikasi 1

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Double Linked List Non Circular (DLLNC) (Head)
=====
1. Tambah Depan
2. Tambah Belakang
3. Tampilkan Data
4. Hapus Depan
5. Hapus Belakang (Hapus ke-N)
6. Hapus Target
7. Reset (Clear Bergantian)
8. Kembali ke Menu
Pilihan : 3
4 6 1 2 3
<press any key to continue>
Double Linked List Non Circular (DLLNC) (Head)
=====
1. Tambah Depan
2. Tambah Belakang
3. Tampilkan Data
4. Hapus Depan
5. Hapus Belakang (Hapus ke-N)
6. Hapus Target
7. Reset (Clear Bergantian)
8. Kembali ke Menu
Pilihan : 
```

Gambar 2 Screenshot Output Program Tugas 2 Modifikasi 2

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Double Linked List Non Circular (DLLNC) (Head)
=====
1. Tambah Depan
2. Tambah Belakang
3. Tampilkan Data
4. Hapus Depan
5. Hapus Belakang (Hapus ke-N)
6. Hapus Target
7. Reset (Clear Bergantian)
8. Kembali ke Menu
Pilihan : 3
4 6 2 3
<press any key to continue>
Double Linked List Non Circular (DLLNC) (Head)
=====
1. Tambah Depan
2. Tambah Belakang
3. Tampilkan Data
4. Hapus Depan
5. Hapus Belakang (Hapus ke-N)
6. Hapus Target
7. Reset (Clear Bergantian)
8. Kembali ke Menu
Pilihan : 
```

Gambar 3 Screenshot Output Program Tugas 2 Modifikasi 3

```
Double Linked List Non Circular (DLLNC) (Head)
=====
1. Tambah Depan
2. Tambah Belakang
3. Tampilkan Data
4. Hapus Depan
5. Hapus Belakang (Hapus ke-N)
6. Hapus Target
7. Reset (Clear Bergantian)
8. Kembali ke Menu
Pilihan : 7
Penghapusan urutan 1 : 4
Penghapusan urutan 2 : 3
Penghapusan urutan 3 : 6
Penghapusan urutan 4 : 2
Seluruh data telah dibersihkan secara bergantian.
~press any key to continue~
Double Linked List Non Circular (DLLNC) (Head)
=====
1. Tambah Depan
2. Tambah Belakang
3. Tampilkan Data
4. Hapus Depan
5. Hapus Belakang (Hapus ke-N)
6. Hapus Target
7. Reset (Clear Bergantian)
8. Kembali ke Menu
Pilihan : █
```

Gambar 4 Screenshot Output Program Tugas 2 Modifikasi 4

LINK GITHUB

<https://github.com/IntraGH/Algoritma-dan-Struktur-Data/tree/main/Tugas%202>